

Руководство по навигации и поиску в исходниках ядра Linux

Table of Contents

1. Введение	2
2. Онлайн-инструменты для навигации	2
2.1. Elixir Bootlin	2
2.1.1. Основные возможности	2
2.2. Kernel.org Documentation	3
3. Локальная работа с исходниками	4
3.1. Установка исходников	4
3.1.1. Ubuntu/Debian	4
3.1.2. Fedora/RHEL	4
3.1.3. Скачивание с kernel.org	4
3.2. Генерация документации	4
3.3. Инструменты навигации	5
3.3.1. ctags для Vim	5
3.3.2. ctags для Emacs	5
3.3.3. VS Code	5
3.3.4. cscope	5
4. Структура документации ядра	6
4.1. Основные разделы	6
4.2. Чтение документации	6
4.2.1. Просмотр reStructuredText файлов	6
4.2.2. Поиск в документации	7
5. Практические методы поиска	7
5.1. Поиск по подсистеме	7
5.2. Поиск по типу функции	7
5.2.1. Функции выделения памяти	7
5.2.2. Функции работы со временем	7
5.2.3. Функции синхронизации	8
5.3. Анализ экспортируемых символов	8
5.4. Поиск в mailing lists	8
6. Чтение kerneldoc	8
6.1. Формат kerneldoc	9
6.2. Извлечение kerneldoc	9
7. Практические примеры	9
7.1. Пример 1: Поиск API для работы с GPIO	9

7.2. Пример 2: Поиск API для работы с памятью	10
7.3. Пример 3: Поиск API для работы со временем	10
8. Полезные ресурсы	10
8.1. Онлайн-ресурсы	10
8.2. Книги	11
8.3. Видео-ресурсы	11
9. Советы и рекомендации	11
9.1. Общие рекомендации	11
9.2. Избегание типичных ошибок	11
9.3. Эффективные рабочие процессы	11
10. Заключение	12
11. Приложение А: Шпаргалка по командам	12
11.1. Поиск в исходниках	12
11.2. Генерация тегов	12
11.3. Работа с документацией	13
12. Приложение В: Полезные алиасы	13

1. Введение

Разработка модулей ядра Linux требует умения эффективно работать с исходным кодом и документацией. Ядро содержит миллионы строк кода, тысячи функций и обширную документацию. Умение быстро находить нужные API, понимать их назначение и правильно использовать — ключевой навык разработчика ядра.

В этом руководстве мы рассмотрим все доступные методы поиска и навигации по исходникам ядра Linux, от онлайн-инструментов до локальных утилит.

2. Онлайн-инструменты для навигации

2.1. Elixir Bootlin

[Elixir Bootlin](#) — это наиболее мощный инструмент для навигации по исходникам ядра Linux. Он предоставляет:

- Полнотекстовый поиск по всем версиям ядра
- Перекрестные ссылки между определениями и использованиями
- Навигацию по структуре каталогов
- Просмотр истории изменений файлов

2.1.1. Основные возможности

Поиск функций

Для поиска функции введите её название в поисковую строку. Например, поиск `ktime_get` покажет:

- **Definitions** — где функция определена (реализация)
- **Declarations** — где функция объявлена (в заголовочных файлах)
- **References** — где функция используется в коде

Навигация по подсистемам

Перейдите в нужный каталог через навигационное меню:

```
source/  
├── arch/           # Архитектурно-зависимый код  
├── drivers/        # Драйверы устройств  
│   ├── usb/        # USB драйверы  
│   ├── spi/         # SPI драйверы  
│   ├── i2c/         # I2C драйверы  
│   └── gpio/        # GPIO драйверы  
├── include/        # Заголовочные файлы  
│   └── linux/       # Основные API ядра  
├── kernel/         # Ядро системы  
├── mm/             # Управление памятью  
└── Documentation/  # Документация
```

Пример использования

1. Перейдите на <https://elixir.bootlin.com/linux/latest/source>
2. Выберите версию ядра (например, `v6.6`)
3. Введите в поиск `ktime_get_real`
4. Изучите:
 - Определение в `include/linux/timekeeping.h`
 - Реализацию в `kernel/time/timekeeping.c`
 - Примеры использования в `drivers/`

2.2. Kernel.org Documentation

Официальная документация ядра доступна по адресу:

<https://www.kernel.org/doc/html/latest/>

Основные разделы:

- **Driver API** — API для разработки драйверов
- **Core API** — основные функции ядра

- **Subsystem APIs** — документация по подсистемам
- **Development Process** — процесс разработки ядра

3. Локальная работа с исходниками

3.1. Установка исходников

3.1.1. Ubuntu/Debian

```
# Установка исходников текущей версии ядра
sudo apt-get install linux-source

# Исходники будут в /usr/src/
cd /usr/src/linux-source-*

# Распаковка (если необходимо)
tar xf linux-source-*.tar.xz
```

3.1.2. Fedora/RHEL

```
sudo dnf install kernel-source
cd /usr/src/kernels/$(uname -r)
```

3.1.3. Скачивание с kernel.org

```
# Скачивание конкретной версии
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz
tar xf linux-6.6.tar.xz
cd linux-6.6
```

3.2. Генерация документации

Ядро содержит встроенную документацию в формате reStructuredText. Для генерации HTML-документации:

```
# Установка зависимостей (Ubuntu/Debian)
sudo apt-get install python3-sphinx python3-sphinx-rtd-theme

# Генерация документации
cd /path/to/linux-source
make htmldocs

# Открытие документации
```

3.3. Инструменты навигации

3.3.1. ctags для Vim

```
# Генерация файла tags
cd /path/to/linux-source
make tags

# Использование в Vim
vim include/linux/timekeeping.h
# Перейти к определению: Ctrl+]
# Вернуться назад: Ctrl+T
# Поиск по тегу: :tag ktime_get
```

3.3.2. ctags для Emacs

```
# Генерация файла TAGS
cd /path/to/linux-source
make TAGS

# Использование в Emacs
emacs include/linux/timekeeping.h
# Перейти к определению: M-.
# Вернуться назад: M-,
```

3.3.3. VS Code

```
# Установка расширения C/C++ от Microsoft
# Открытие папки с исходниками ядра
code /path/to/linux-source

# VS Code автоматически создаст IntelliSense базу
# Ctrl+Click на функции → переход к определению
# F12 → переход к определению
# Shift+F12 → найти все использования
```

3.3.4. cscope

```
# Генерация базы cscope
cd /path/to/linux-source
make cscope
```

```
# Использование
cscope -d
# В интерактивном меню:
# 1. Найти это определение C-символа
# 2. Найти где этот символ используется
# 3. Найти вызовы этой функции
```

4. Структура документации ядра

4.1. Основные разделы

Документация ядра организована следующим образом:

```
Documentation/
├── driver-api/           # API для драйверов устройств
│   ├── basics.rst       # Основы написания драйверов
│   ├── usb/             # USB подсистема
│   ├── pci/             # PCI подсистема
│   ├── spi/             # SPI подсистема
│   ├── i2c/             # I2C подсистема
│   ├── gpio/            # GPIO подсистема
│   └── input/           # Input устройства
├── core-api/            # Основные API ядра
│   ├── kernel-api.rst   # API ядра
│   ├── memory-allocation.rst # Выделение памяти
│   └── timekeeping.rst  # Работа со временем
├── admin-guide/         # Руководство администратора
├── userspace-api/       # API пользовательского пространства
└── dev-tools/           # Инструменты разработки
```

4.2. Чтение документации

4.2.1. Просмотр reStructuredText файлов

```
# Установка утилиты для просмотра rst
sudo apt-get install python3-docutils

# Конвертация в HTML
rst2html Documentation/core-api/timekeeping.rst > timekeeping.html

# Или использование pandoc
pandoc Documentation/core-api/timekeeping.rst -o timekeeping.html
```

4.2.2. Поиск в документации

```
# Поиск по ключевым словам
grep -r "ktime" Documentation/ | grep "\.rst:"

# Поиск в конкретном разделе
grep -r "timekeeping" Documentation/core-api/

# Поиск с контекстом
grep -B2 -A5 "ktime_get" Documentation/core-api/timekeeping.rst
```

5. Практические методы поиска

5.1. Поиск по подсистеме

Если вы пишете драйвер для конкретной подсистемы, изучите существующие драйверы:

```
# Список SPI драйверов
ls drivers/spi/

# Изучение простого драйвера
cat drivers/spi/spi-bitbang.c | head -100

# Поиск использования конкретных API
grep -r "spi_transfer" drivers/spi/ | head -20
```

5.2. Поиск по типу функции

5.2.1. Функции выделения памяти

```
# Поиск в заголовочных файлах
grep -r "^void \*kmalloc" include/linux/

# Поиск всех вариантов kmalloc
grep -r "kmalloc\|kzalloc\|kcalloc" include/linux/slab.h

# Примеры использования
grep -r "kmalloc.*GFP_KERNEL" drivers/ | head -10
```

5.2.2. Функции работы со временем

```
# Поиск функций ktime
grep -r "^ktime_t ktime_" include/linux/timekeeping.h
```

```
# Поиск всех ktime функций
grep -r "ktime_" include/linux/ktime.h | grep "static inline"
```

5.2.3. Функции синхронизации

```
# Поиск мьютексов
grep -r "mutex_lock\|mutex_unlock" include/linux/mutex.h

# Поиск спинлоков
grep -r "spin_lock\|spin_unlock" include/linux/spinlock.h

# Поиск семафоров
grep -r "down\|up" include/linux/semaphore.h
```

5.3. Анализ экспортируемых символов

```
# Поиск всех экспортируемых символов в модуле
nm /lib/modules/$(uname -r)/kernel/drivers/usb/core/usbcore.ko | grep " T "

# Поиск конкретной функции
nm /lib/modules/$(uname -r)/kernel/drivers/usb/core/usbcore.ko | grep usb_register

# Просмотр информации о модуле
modinfo /lib/modules/$(uname -r)/kernel/drivers/usb/core/usbcore.ko
```

5.4. Поиск в mailing lists

Обсуждения новых API и изменений часто происходят в mailing lists:

```
# Поиск на lore.kernel.org
# https://lore.kernel.org/

# Поиск по функции
# Введите в поиск: "ktime_get_real"

# Поиск по теме
# Введите: "subject:timekeeping API"
```

6. Чтение kerneldoc

6.1. Формат kerneldoc

Многие функции ядра имеют встроенную документацию:

```
/**
 * ktime_get_real - get the real (wall-) time in ktime_t
 *
 * Returns the real time since Unix epoch (1970-01-01 00:00:00 UTC).
 * This is similar to gettimeofday() in user space.
 *
 * WARNING: This clock can jump backwards due to NTP adjustments!
 * Do not use it for measuring time intervals!
 *
 * Return: ktime_t value representing real time.
 */
ktime_t ktime_get_real(void)
{
    // ...
}
```

6.2. Извлечение kerneldoc

```
# Использование скрипта kernel-doc
scripts/kernel-doc -rst include/linux/timekeeping.h | less

# Генерация документации для конкретной функции
scripts/kernel-doc -rst -function ktime_get_real include/linux/timekeeping.h

# Генерация всей документации
scripts/kernel-doc -rst include/linux/timekeeping.h > timekeeping.rst
```

7. Практические примеры

7.1. Пример 1: Поиск API для работы с GPIO

1. Определите подсистему: GPIO
2. Откройте документацию: [Documentation/driver-api/gpio/](#)
3. Изучите заголовочный файл: [include/linux/gpio.h](#)
4. Найдите примеры использования:

```
grep -r "gpio_request\|gpiod_get" drivers/gpio/ | head -20
```

5. Изучите простой драйвер GPIO:

```
cat drivers/gpio/gpio-mockup.c
```

7.2. Пример 2: Поиск API для работы с памятью

1. Откройте документацию: [Documentation/core-api/memory-allocation.rst](#)
2. Изучите заголовочный файл: [include/linux/slab.h](#)
3. Найдите функции выделения памяти:

```
grep -r "^void \*kmalloc\|^void \*kzalloc" include/linux/slab.h
```

4. Изучите примеры использования:

```
grep -r "kmalloc.*GFP_KERNEL" drivers/usb/ | head -10
```

7.3. Пример 3: Поиск API для работы со временем

1. Откройте документацию: [Documentation/core-api/timekeeping.rst](#)
2. Изучите заголовочные файлы:

```
cat include/linux/timekeeping.h | grep "ktime_t ktime_"  
cat include/linux/ktime.h | grep "static inline"
```

3. Найдите примеры использования:

```
grep -r "ktime_get\|ktime_to_ns" drivers/ | head -20
```

8. Полезные ресурсы

8.1. Онлайн-ресурсы

- [Elixir Bootlin](#) — навигация по исходникам
- [Kernel Documentation](#) — официальная документация
- [LWN.net](#) — статьи и анализ
- [Lore Kernel Mailing List Archive](#) — архив mailing lists
- [Kernel Newbies](#) — ресурсы для начинающих

8.2. Книги

- **Linux Device Drivers, 3rd Edition** — Jonathan Corbet, Alessandro Rubini, Greg Kroah-Hartman
- **Linux Kernel Development, 3rd Edition** — Robert Love
- **Professional Linux Kernel Architecture** — Wolfgang Mauerer
- **Understanding the Linux Kernel** — Daniel P. Bovet, Marco Cesati

8.3. Видео-ресурсы

- Linux Plumbers Conference — доклады разработчиков ядра
- Kernel Recipes — практические советы
- Linux Foundation Training — обучающие курсы

9. Советы и рекомендации

9.1. Общие рекомендации



- Всегда проверяйте версию ядра, для которой написана документация
- Изучайте несколько примеров драйверов, а не только один
- Используйте `git log` для понимания истории изменений API
- Читайте комментарии в коде — они часто содержат важную информацию

9.2. Избегание типичных ошибок



- Не используйте устаревшие API (проверяйте `deprecated` в документации)
- Не копируйте код из старых версий ядра без проверки актуальности
- Не игнорируйте предупреждения компилятора — они часто указывают на проблемы
- Всегда проверяйте возвращаемые значения функций

9.3. Эффективные рабочие процессы

1. **Начинайте с документации** — прочитайте соответствующий раздел в `Documentation/`
2. **Изучайте примеры** — найдите 2-3 простых драйвера в вашей подсистеме
3. **Используйте Elixir Bootlin** — для быстрого поиска и навигации
4. **Читайте kerneldoc** — встроенная документация часто содержит важную информацию
5. **Тестируйте на реальных устройствах** — эмуляторы не всегда корректно воспроизводят

10. Заключение

Эффективная работа с документацией и исходниками ядра Linux — это навык, который развивается с опытом. Используйте комбинацию онлайн-инструментов, локальной навигации и изучения примеров для быстрого освоения новых API.

Помните: ядро Linux разрабатывается сообществом, и всегда есть готовые решения для большинства задач. Ваша цель — найти их и правильно использовать, а не изобретать велосипед.



Если вы застряли или не можете найти нужную информацию:

- Задайте вопрос в mailing list соответствующей подсистемы
- Поищите ответ на Stack Overflow с тегом `linux-kernel`
- Проверьте архивы `lore.kernel.org`
- Обратитесь к сообществу на IRC (канал `#kernel-newbies` на Libera.Chat)

11. Приложение А: Шпаргалка по командам

11.1. Поиск в исходниках

```
# Поиск функции в заголовочных файлах
grep -r "function_name" include/linux/ | grep "\.h:"

# Поиск использования функции
grep -r "function_name" drivers/ | head -20

# Поиск с контекстом
grep -B2 -A5 "function_name" file.c

# Поиск всех функций, начинающихся с префикса
grep -r "^prefix_" include/linux/ | grep "\.h:"
```

11.2. Генерация тегов

```
# Для Vim
make tags

# Для Emacs
make TAGS
```

```
# Для cscope
make cscope
```

11.3. Работа с документацией

```
# Генерация HTML документации
make htmldocs

# Извлечение kernel-doc
scripts/kernel-doc -rst file.h

# Поиск в документации
grep -r "keyword" Documentation/
```

12. Приложение В: Полезные алиасы

Добавьте эти алиасы в ваш `~/.bashrc` или `~/.zshrc`:

```
# Быстрый переход к исходникам ядра
alias ksrc='cd /path/to/linux-source'

# Поиск функции в ядре
alias kgrep='grep -r --include="*.h" --include="*.c"'

# Открытие Elixir Bootlin
alias elixir='firefox https://elixir.bootlin.com/linux/latest/source'

# Открытие документации ядра
alias kdoc='firefox https://www.kernel.org/doc/html/latest/'

# Поиск kernel-doc
alias kdoc-extract='scripts/kernel-doc -rst'
```